

CS & IT ENGINEERING



COMPUTER ORGANIZATION AND ARCHITECTURE

Cache Organization

Lecture No.- 8

By- Vishvadeep Gothi sir



Recap of Previous Lecture



Topic

Miss Penalty

Topic

Types of Misses

Topic

Cache Mapping Hardware

Topics to be Covered

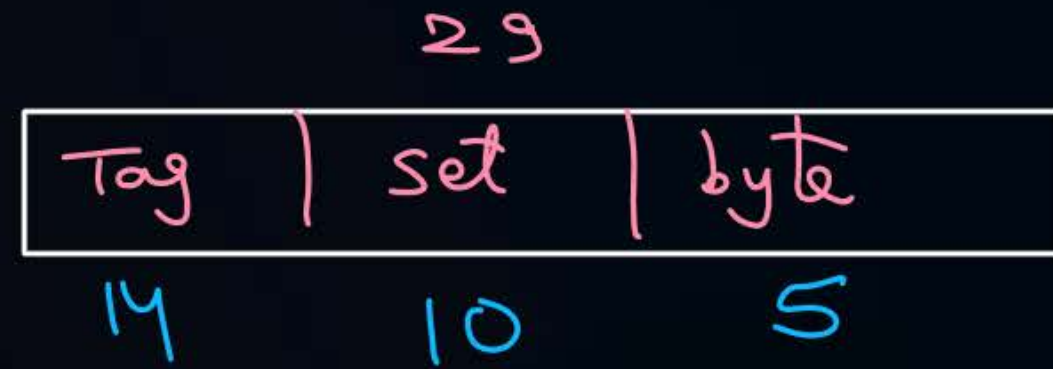


Topic

Array Access With Cache



#Q. Cache Size = 128KB
Block size = 32 bytes
Main memory address = 29-bits
4-way set associative cache



1. Tag size? 14 bits
2. Tag Directory size? $2^{12} * 14$ bits
3. Comparator required? 4 14-bits comparators
4. MUX required? $4 * 14 = 56$
Size = $2^{10} : 1$
= 1024 : 1

#Q. Consider two cache organizations. First one is 32 KB 2-way set associative with 32-bytes block size, the second is of same size but direct mapped. The size of an address is 32 bits in both cases. A 2-to-1 multiplexer has latency of 0.6 ns while a k-bit comparator has latency of $\frac{k}{10}$ ns. The hit latency of the set associative organization is h_1 while that of direct mapped is h_2 .

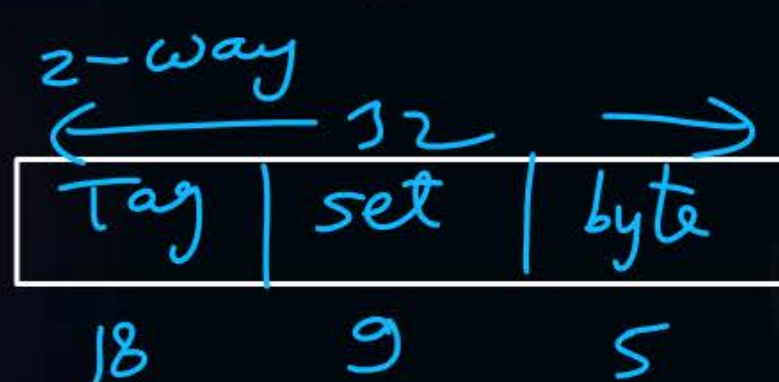
The value of h_1 is:

A ✓ 2.4 ns

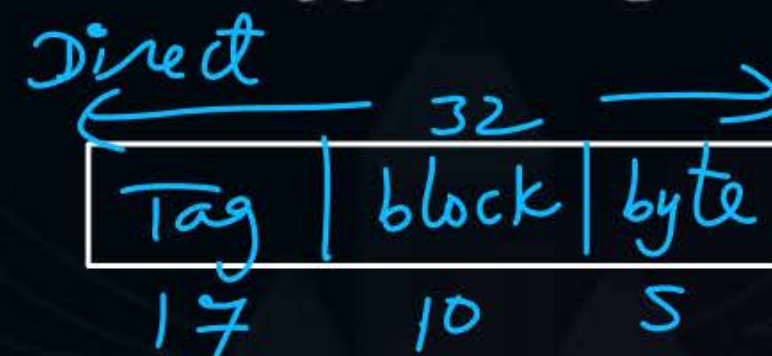
B 2.3 ns

C 1.8 ns

D 1.7 ns



$$0 + \frac{18}{10} + 0.6 = 2.4 \text{ ns}$$



$$0 + \frac{17}{10} = 1.7 \text{ ns}$$

#Q. Consider two cache organizations. First one is 32 KB 2-way set associative with 32-bytes block size, the second is of same size but direct mapped. The size of an address is 32 bits in both cases. A 2-to-1 multiplexer has latency of 0.6 ns while a k-bit comparator has latency of $\frac{k}{10}$ ns. The hit latency of the set associative organization is h_1 while that of direct mapped is h_2 .

The value of h_2 is:

- A** 2.4 ns
- B** 2.3 ns
- C** 1.8 ns
- D** ✓ 1.7 ns



Topic : Questions on Cache and Array

- Direct mapping
- Cache Size = 32 bytes
- Block size = 4 bytes
- Array in main memory A[40], each element is 1 byte

$$\left. \begin{array}{l} \text{Cache Size} = 32 \text{ bytes} \\ \text{Block size} = 4 \text{ bytes} \end{array} \right\} \text{no. of blocks in cache} = \frac{32 \text{ B}}{4 \text{ B}} = 8$$

$$\text{size of array} = 40 \text{ elements} = 40 * 1 = 40 \text{ Bytes}$$

$$\text{no. of blocks needed to store array in mm} = \frac{40 \text{ B}}{4 \text{ B}} = 10 \text{ blocks}$$

$$\text{no. of array elements in one block} = \frac{4 \text{ B}}{1 \text{ B}} = 4$$

cache

0	$A[24]$		
	$A[27]$		
1	$A[28]$		
	$A[31]$		
2	$A[0]$	$A[32]$	$A[0]$ $A[32]$
	$A[3]$	$A[35]$	$A[3]$ $A[35]$
3	$A[4]$	$A[36]$	$A[4]$ $A[36]$
	$A[7]$	$A[39]$	$A[7]$ $A[39]$
4	$A[8]$		
	$A[11]$		
5	$A[12]$		
	$A[15]$		
6	$A[16]$		
	$A[19]$		
7	$A[20]$		
	$A[23]$		

၈၈၇

$A[0]$

$A[1]$

$A[2]$

$A[3]$

$A[39]$

mm
block
no. = 2

10 blocks

Access of element	hit/miss
A[0]	miss
A[1]	hit
A[2]	hit
A[3]	hit
A[4]	miss
A[5] to A[7]	hit, hit, hit
A[8]	miss
A[9]... A[11]	hit, hit, hit
A[12]	miss
A[13]... A[15]	hit
A[16]	miss
A[17]... A[19]	hit
A[20]	miss
A[21]... A[23]	hit
A[24]	miss
A[25] - ... A[27]	hit

element access	hit miss
A[28]	miss
A[29]... A[31]	hit
A[32]	miss
A[33]... A[35]	hit
A[36]	miss
A[37]... A[39]	hit

first access of array,

no. of miss = 10

no. of hits = $40 - 10 = 30$

↑
40 elements hence 40 mem. accesses

2nd access of array :-

Access	hit/miss
A[0]	miss
A[1] ~ A[3]	hit
A[4]	miss
A[5] ~ A[7]	hit
A[8] ~ A[31]	hit
A[32]	miss
A[33] ~ A[35]	hit
A[36]	miss
A[37] ~ A[39]	hit

2nd access of array , } no. of hits = 40 - 4
no. of miss = 4 } = 36

3rd access of array
no. of miss = 4 hits = 36

If CPU accesses entire array 3 times

$$\text{hit ratio} = \frac{30 + 36 + 36}{3 \times 40} \\ = 0.85 \text{ or } 85\%$$

$$\begin{aligned}
 \text{no. of miss in 1st access} &= \text{no. of blocks to store array} \\
 \text{---||---} \quad 2^{\text{nd}} \text{ ---||---} &= 2 * \left(\begin{array}{l} \text{no. of blocks to store array} \\ \text{--- no. of blocks in cache} \end{array} \right)
 \end{aligned}$$

ex:- (2)

array size = 80 elements

array stored on memory on = 20 blocks

Cache has no. of blocks = 16 blocks

if array is accessed 4 times, hit ratio = — ?

no. of miss in 1st access = 20

— || — 2nd — || — = $2 * (20 - 16) = 8$

— || — 3rd — || — = 8

— || — 4th — || — = 8

Total miss = 44

no. of hits = $(80 * 4) - 44$
= 276

hit ratio = $\frac{276}{320} = 0.8625$
Ans.

#Q. Consider a machine with a byte addressable main memory of 2^{16} bytes. Assume that a direct mapped data cache consisting of 32 lines of 64 bytes each is used in the system. A 50×50 two-dimensional array of bytes is stored in the main memory starting from memory location 1100H. Assume that the data cache is initially empty. The complete array is accessed twice. Assume that the contents of the data cache do not change in between the two accesses.

How many data cache misses will occur in total?

#Q. Which of the following lines of the data cache will be replaced by new blocks in accessing the array for the second time?

- A** line 4 to line 11
- B** line 4 to line 12
- C** line 0 to line 7
- D** line 0 to line 8



#Q. Consider a direct mapped write back data cache of size 2KB with block size of 128 bytes. The cache is considered to be empty initially. The byte addressable main memory has size 1Mbytes. Further consider that there is an array $A[30][25]$ with each element occupies 4 bytes. The base address of array is $(1A300)_{16}$. The array is accessed 4 times. And between the accesses, there is no any data cache changes happen. Hit ratio of cache for the array access is?



Topic : Questions on Cache and Array

- Cache Size = 64 bytes
- Block size = 8 bytes
- Array in main memory int A[8][8], each element is 2 bytes

array size = $8 * 8 = 64$ elements = $64 * 2 = 128$ bytes

no. of blocks to store array in mem = $\frac{128 \text{ B}}{8 \text{ B}} = 16$ blocks

no. of array elements in single block = $\frac{8 \text{ B}}{2 \text{ B}} = 4$

no. of blocks to store row = $\frac{8}{4} = 2$

Cache	
0	$A[3][0]$ $A[3][3]$
1	
2	$A[0][0]$ $A[4][0]$ $A[0][3]$ $A[4][3]$
3	
4	$A[1][0]$ $A[5][0]$ $A[1][3]$ $A[5][3]$
5	
6	$A[2][0]$ $A[2][3]$
7	

Memory	
2	$A[0][0]$ $A[0][1]$ $A[0][2]$ $A[0][3]$
3	$\vdots$ $A[0][7]$
4	$A[1][0]$ $\vdots$ $A[1][7]$
	$\vdots$
	$A[7][0]$ $\vdots$ $A[7][7]$

array	
	0 1 2 3 4 5 6 7
0	$A[0][0]$
1	$A[1][0]$
2	$A[2][0]$
3	
4	
5	
6	
7	$A[7][7]$

2-d array is stored in row-major order in memory.

Access of array row-wise	hit & miss
A[0][0]	miss
A[0][1]	hit
A[0][2]	
A[0][3]	miss
A[0][4]	
A[0][5]	
A[0][6]	

first access of array,
no. of miss = no. of blocks to
store array = 16

Access of array column-wise	hit or miss
A[0][0]	miss
A[1][0]	miss
A[2][0]	miss
A[3][0]	miss
A[4][0]	miss
A[5][0]	miss

↓
no. of miss = no. of elements in array
= 64

#Q. A CPU has a 32KB direct mapped cache with 128 byte-block size. Suppose A is two-dimensional array of size 512×512 with elements that occupy 8-bytes each. Consider the following two C code segments, P1 and P2.

P1:

```
for (i = 0 ; i < 512 ; i++)  
{  
    for (j = 0 ; j < 512 ; j++)  
    {  
        x += A[i][j] ;  
    }  
}
```

P2:

```
for (i = 0 ; i < 512 ; i++)  
{  
    for (j = 0 ; j < 512 ; j++)  
    {  
        x += A[j][i] ;  
    }  
}
```

#Q. P1 and P2 are executed independently with the same initial state, namely, the array A is not in the cache and i, j, x are in registers. Let the number of cache misses experienced by P_1 be M_1 and that for P_2 be M_2 .

The value of M_1 is :



#Q. P1 and P2 are executed independently with the same initial state, namely, the array A is not in the cache and i, j, x are in registers. Let the number of cache misses experienced by P_1 be M_1 and that for P_2 be M_2 .

The value of M_2 is :





2 mins Summary



Topic

Array Access With Cache





Happy Learning

THANK - YOU

